Correlation of Object Detection Performance with Visual Saliency and Depth Estimation

Team: CAP-C6-Group 6
Team Members: Saranbabu, Hemanth, Bhamiesha and Anusha kankan

Project Repository: https://github.com/saransvijay/Project_UOA

1. Abstract

Accurate object detection remains a critical challenge in computer vision, particularly in complex scenes involving occlusion, scale variation, and small objects. Although modern deep learning models achieve high detection accuracy, understanding how complementary visual tasks contribute to detection performance remains an important research direction. Human visual perception relies on attention and spatial reasoning, motivating the exploration of auxiliary tasks such as visual saliency and depth estimation.

Previous research has explored multi-task learning approaches that combine object detection with saliency or depth estimation, but these studies mainly evaluate end-to-end performance rather than analysing the underlying relationships between tasks. Bartolo et al. (1) investigated correlations between object detection accuracy, visual saliency, and depth estimation using offline experiments and annotated datasets, showing stronger correlations between saliency and detection performance.

The **VisualCue Framework** extends that analysis by introducing a real-time, annotation-free framework integrating RT-DETR (3) for object detection, Score-CAM (4) for attention mapping, the Segment Anything Model (SAM) (2) for object segmentation, and Zoe Depth (5) for monocular depth estimation. Experiments conducted on the COCO 2017 unlabelled dataset evaluate spatial correlations between attention maps, segmentation masks, and depth representations. Results show stronger alignment between model attention and segmented object regions than with depth cues, supporting the effectiveness of the framework for practical computer vision applications.

2. Introduction

Object detection is a key component of modern computer vision systems used in autonomous driving, surveillance, robotics, and healthcare. Despite significant improvements in detection accuracy, real-world challenges such as occlusion, small objects, and varying lighting conditions make it difficult for models to consistently focus on the correct regions.

In real-time applications, high detection accuracy alone is not sufficient. Predictions must be based on meaningful visual information rather than background noise. Analysing the relationship between object detection, spatial attention, and scene structure provides deeper insight into model behaviour, supporting the development of more reliable computer vision solutions.

3. Literature Review:

This project is primarily inspired by Bartolo et al. (2024) (1), who analysed the relationship between object detection performance, visual saliency, and depth estimation using offline experiments and annotated datasets. While their study provides valuable insights, it is limited to non-real-time analysis and depends on predefined saliency models and ground-truth annotations. To overcome these limitations, the VisualCue Framework integrates four key components: RT-DETR (Zhao et al., 2024) (3) for real-time object detection, Score-CAM (Wang et al., 2020) (4) for stable, gradient-free attention mapping, SAM (Kirillov et al., 2023) (2) for annotation-free segmentation, and ZoeDepth (2023) (5) for robust monocular depth estimation across diverse environments.

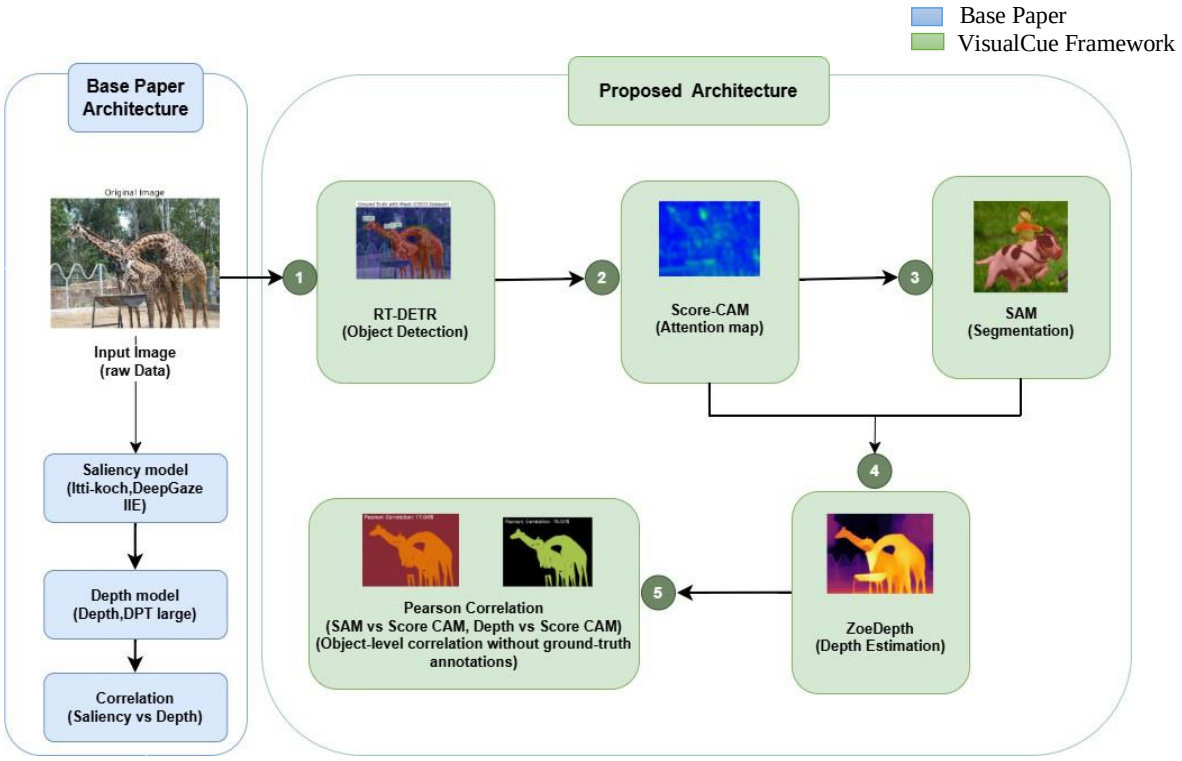
4. Materials and Method:

The VisualCue Framework shown in **Figure 1** is a real-time, annotation-free pipeline that integrates object detection, attention mapping, segmentation, and depth estimation into a unified end-to-end system, enabling analysis on unlabelled data beyond prior offline studies.

4.1 Dataset

The framework was evaluated on the COCO 2017 Unlabelled Dataset, a large-scale benchmark widely used in computer vision research. A subset of 100 images spanning 15 object categories was selected for analysis: Airplane, Bed, Elephant, Giraffe, Pizza, Person, Motorcycle, Parking Meter, Horse, Stop Sign, Zebra, Bear, Cake, Teddy Bear, and Hot Dog. These categories represent a diverse range of object shapes, sizes, and visual complexity. For full dataset details, refer to Appendix A.

Figure 1: VisualCue Framework - Pipeline Architecture



The pipeline consists of four components: (A) RT-DETR - Object Detection, (B) Score-CAM - Attention Mapping, (C) SAM - Segmentation, and (D) ZoeDepth - Depth Estimation, followed by Pearson Correlation Analysis.

Component A: RT-DETR (Object Detection) performs real-time object detection using RT-DETR. The input image is processed to generate bounding boxes and associated confidence scores. These bounding boxes serve as structural anchors for subsequent stages, ensuring object-level consistency throughout the pipeline.

Component B: Score-CAM (Attention Mapping) interpretability is incorporated by applying Score-CAM to the detection model. This produces attention maps that highlight spatial regions contributing most strongly to detection decisions. Unlike gradient-based saliency techniques, Score-CAM generates class-discriminative attention maps without modifying gradients, thereby improving stability and interpretability.

Component C: SAM (Segmentation) performs object-level segmentation using the Segment Anything Model. Bounding boxes from RT-DETR are used as prompts to guide SAM in producing precise segmentation masks, eliminating the dependence on manually annotated masks and allowing the system to operate effectively on unlabelled datasets.

Component D: ZoeDepth (Depth Estimation) introduces spatial reasoning through depth estimation. ZoeDepth is applied to the input image to produce a dense monocular depth map, from which object-specific depth values are extracted using SAM segmentation masks, enabling analysis of how spatial distance relates to model attention and detection confidence.

Component E: Pearson Correlation Analysis computes Pearson correlation coefficients to measure relationships between attention intensity, segmentation regions, detection confidence, and depth values. Chosen for its interpretability and efficiency, it measures linear associations between spatial

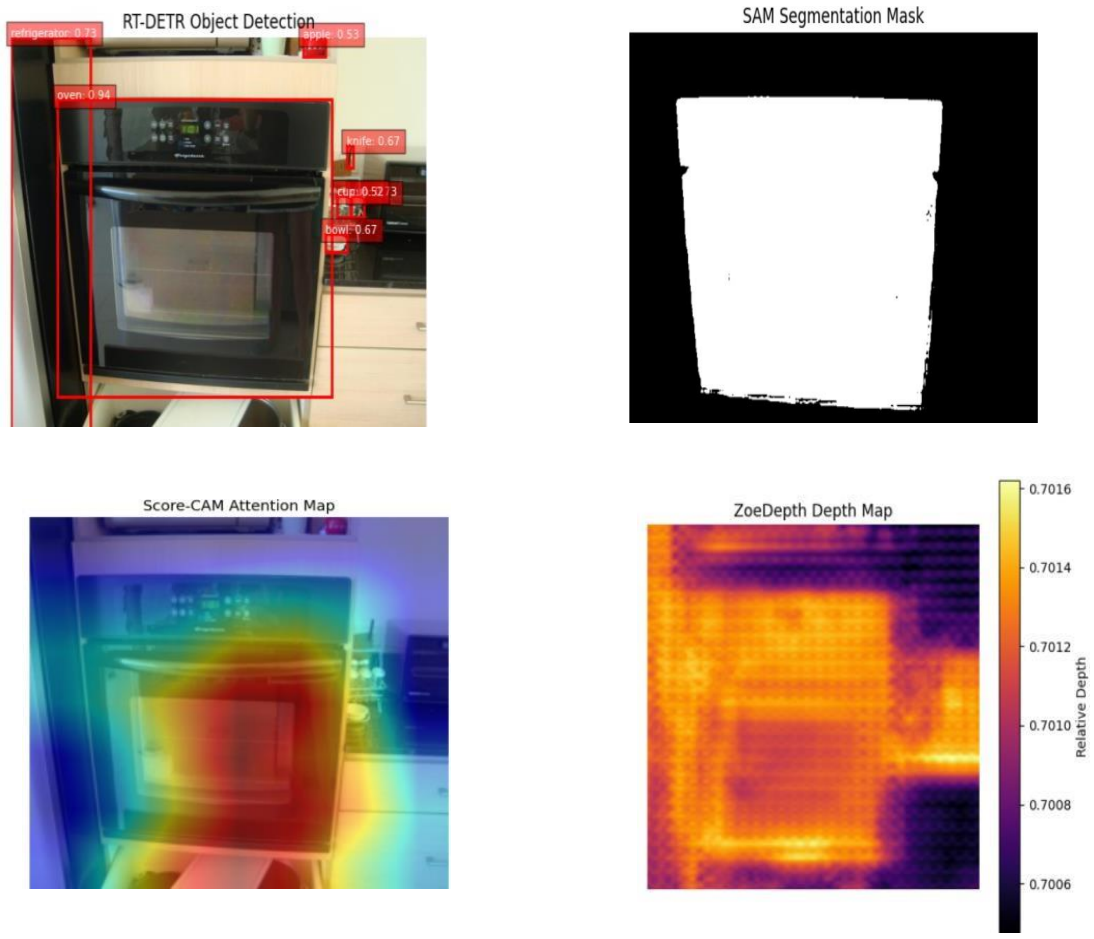
representations, maintaining continuity with the base paper while extending it into a real-time, automated framework.

4.2 Experimental Setup

The **Figure 2** shows sample outputs from the VisualCue Framework, including object detections, attention maps, segmentation masks, and depth maps for the same input image. The results demonstrate consistent alignment between model attention and object regions, confirming the effectiveness of the pipeline.

Figure 2: Representative Pipeline Results for a Sample Image

(a) RT-DETR Detection Output (First row left) - bounding boxes drawn around detected objects with confidence scores. (b) SAM Segmentation Mask (First row right) - white regions represent detected object areas; black regions are background. (c) Score-CAM Attention Map (Second row left) - a heatmap where warm colours (red/yellow) indicate regions the model focuses on most strongly, and cool colours (blue) indicate low attention. (d) ZoeDepth Depth Map (Second row right) - colours represent relative depth: colours like red and orange indicate regions closer to the camera, while colours like blue and purple indicate regions further away. The colour bar on the right shows the corresponding depth scale.



5. Results:

5.1 Graphical interface

The following screenshots in the **Figure 3** illustrate the web-based graphical interface of the framework, demonstrating the end-to-end pipeline on sample inputs.

Figure 3: VisualCue Framework — Graphical User Interface

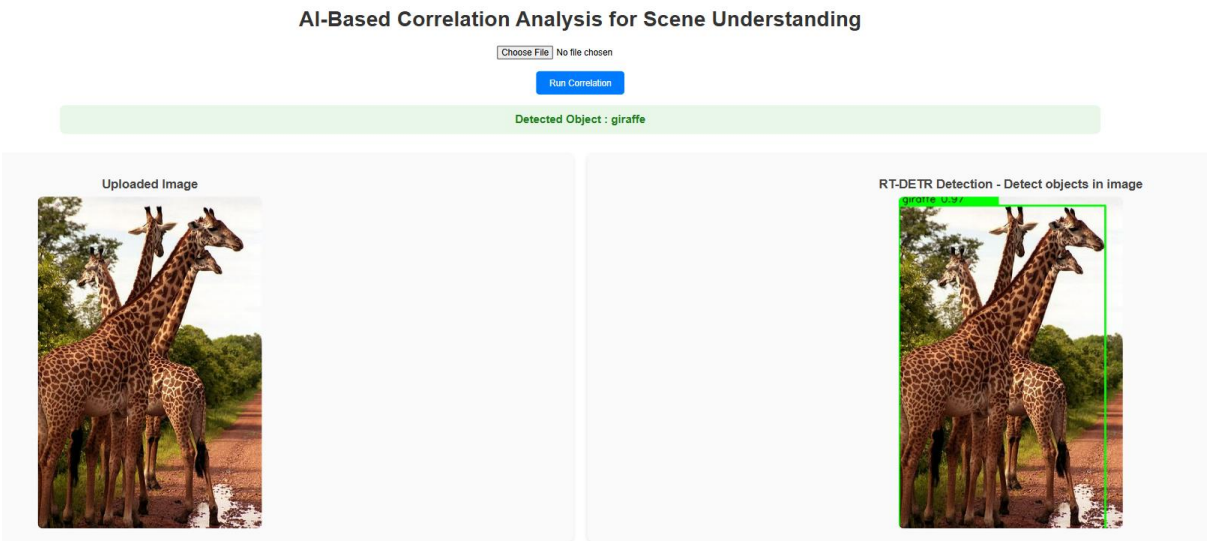


Figure 3a: Original Image (left) and RT-DETR Object Detection (Right) - The framework draws green bounding boxes around all detected objects in the image, showing that the model has successfully located it. The label at the top confirms the detected object class.

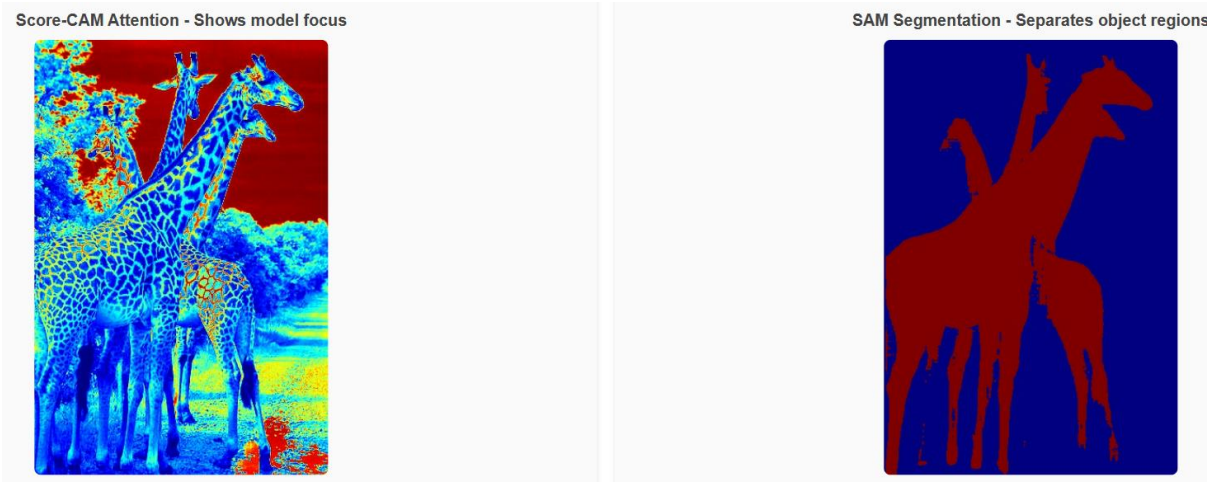


Figure 3b: Score-CAM attention map (left) and SAM segmentation mask (right), highlighting model focus regions and object boundaries

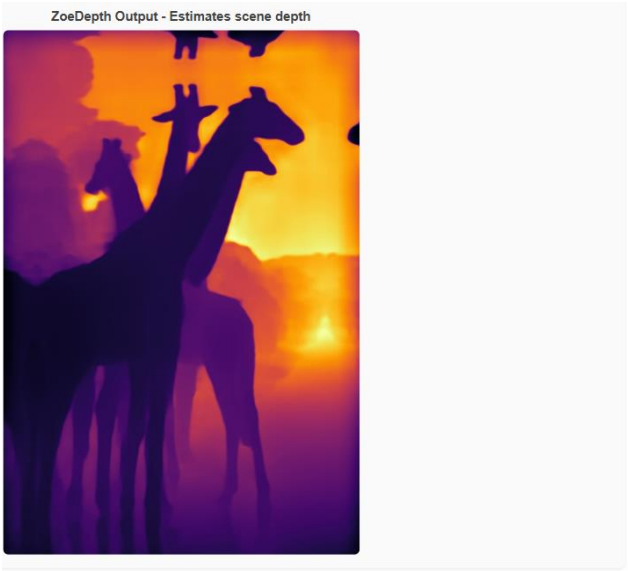


Figure 3c: This map estimates how far each part of the scene is from the camera.
The correlation score of 0.434 shown at the bottom indicates the degree of alignment between the model's attention and the depth distribution.

5.2 Evaluation Methodology

The VisualCue Framework achieves the highest Mean Pearson Correlation of 0.186 (SAM & Score-CAM), outperforming all baseline techniques including the best base model DeepGaze IIE (0.17), indicating better alignment between model attention and actual object regions. The attention-depth correlation of 0.143 further confirms stable spatial understanding within a real-time, annotation-free architecture. As shown in **Table 1**, these results highlight the novelty of transforming prior offline correlation analysis into a unified, automated, and deployable system. The primary evaluation metric, Pearson Correlation (ρ), measures the linear relationship between two spatial maps — such as an attention map and a segmentation mask — as shown in Equation 1.

$$\rho_{X,Y} = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y}$$

Equation 1: Pearson Correlation Coefficient

The value of ρ ranges from −1 to +1, where +1 indicates perfect positive correlation, 0 indicates no correlation, and −1 indicates perfect negative correlation. In this study, all reported values are positive, indicating a consistent alignment between the compared spatial maps.

Table 1: Comparison of Mean Pearson Correlation - Base Paper Models vs VisualCue Framework
A higher Mean Pearson Correlation value indicates stronger alignment. The best result is highlighted in bold.

Technique	Mean Pearson Correlation	Model Type
Depth Anything	0.125	Depth prediction
DPT-Large	0.129	Depth prediction
Itti-Koch Model	0.13	Saliency prediction
DeepGaze IIE	0.17	Saliency prediction
VisualCue Framework (ZoeDepth & Score CAM)	0.143	Attention-Depth
VisualCue Framework (SAM & Score CAM)	**0.186**	Attention-Segmentation

5.4 Category Metrics

Table 2 compares the Mean Pearson Correlation across 15 object categories between the base paper models and the VisualCue Framework. Notably, while the base paper relied on pre-annotated datasets with ground-truth labels, the VisualCue Framework operates entirely on unlabelled data through its automated pipeline. Despite this fundamental difference, the VisualCue Framework achieves consistently higher correlation values across all 15 categories, demonstrating that annotation-free analysis can match and outperform traditional annotated methods.

Table 2: Category-Level Pearson Correlation - Base Model vs. VisualCue Framework
A higher value indicates stronger alignment between model attention and object regions. The VisualCue Framework column reflects annotation-free results.

Category Level Metrics			
Category Name	Base Model	VisualCue Framework	No. of Images analysed
Airplane	0.326	0.388	100
Bed	0.259	0.287	100
Elephant	0.301	0.373	100
Giraffe	0.326	0.425	100
Pizza	0.228	0.303	100
Person	0.241	0.32	100
Motorcycle	0.254	0.312	100
parking meter	0.304	0.355	97
Horse	0.291	0.345	100
stop sign	0.288	0.335	100
Zebra	0.28	0.347	100
Bear	0.279	0.345	100
Cake	0.256	0.312	100
teddy bear	0.243	0.292	100
hot dog	0.243	0.297	100

6. Implementation and User Benefit:

This project focuses on quantitative evaluation of object detection performance by analysing relationships between detection confidence, attention mechanisms, and depth information. While the findings provide insights applicable to real-time deployment, direct end-user benefits remain exploratory and will require further validation.

7. Limitations and Further Improvements:

This framework establishes a real-time, annotation-free pipeline evaluated on a focused dataset to validate feasibility. Broader experimentation across diverse scenarios will further enhance robustness, and future work may explore advanced modelling approaches and large-scale deployment settings.

8. Bibliography and References:

(1) Bartolo et al., 2024 - Correlation of Object Detection Performance with Visual Saliency and Depth Estimation. arXiv: https://arxiv.org/abs/2411.02844

(2) Kirillov et al., 2023 - Segment Anything (SAM). arXiv: https://arxiv.org/abs/2304.02643

(3) Zhao et al., 2024 -DETRs Beat YOLOs on Real-Time Object Detection.
arXiv: https://arxiv.org/abs/2304.08069

(4) H. Wang *et al.*, "Score-CAM: Score-Weighted Visual Explanations for Convolutional Neural Networks," *arXiv:1910.01279 [cs]*, Apr. 2020
arXiv: https://arxiv.org/abs/1910.01279

(5) S. F. Bhat, R. Birkl, D. Wofk, P. Wonka, and M. Müller, "Zoe Depth: Zero-shot Transfer by Combining Relative and Metric Depth," *arXiv.org*, Feb. 23, 2023.
arXiv: https://arxiv.org/abs/2302.12288

9. Individual Progress:

Task	Sub-task	Owner	Jira Board Ticket / Link
Task 1: Dataset Preparation & Preprocessing	Collect COCO Unlabeled-2017 dataset	Hemanth	https://app.clickup.com/t/86d18xuvv
	Preprocess images (resize, normalize, organize folder structure)	Hemanth	https://app.clickup.com/t/86d18xv7n
Task 2: Object De-tection Module	Implement RT-DETR for real-time object detection	Hemanth	https://app.clickup.com/t/86d18xvf1
	Export bounding boxes for SAM Prompting	Bhamiesha	https://app.clickup.com/t/86d18xvn7
Task 3: Segmentation Module	Integrate SAM to generate object masks using RT-DETR outputs	Bhamiesha	https://app.clickup.com/t/86d18xvrf
Task 4: Attention Map Module	Implement Score-CAM to compute model attention regions	Saran	https://app.clickup.com/t/86d18xvuw
Task 5: Depth Es-timation Module	Integrate Zoe Depth for per-object depth Prediction	Saran	https://app.clickup.com/t/86d18xvwh
Task 6: Correlation Analysis Module	Compute correlation between Score-CAM vs SAM, and Score-CAM vs Depth maps	Anusha	https://app.clickup.com/t/86d18xvxk
Task 7: System In-tegration	Combine all modules into a unified pipeline	Anusha & Saran	https://app.clickup.com/t/86d18xw39
Task 8: Final Re-porting & Evaluation	Prepare documentation, results, graphs, and final report	All team members	https://app.clickup.com/t/86d18xwj5

9.1 Parallel Work & Dependency Notes

Task 1 must be completed before **Tasks 2–5** start.

Task 2 (RT-DETR) runs in parallel with **Task 4 (Score-CAM)** since both use the same input image.

Task 2 output (bounding boxes) is required for **Task 3 (SAM segmentation).**

Task 3 & Task 5 output maps that feed into **Task 6 (Correlation Analysis).**

Task 7 depends on completion of **Tasks 2–6**.

Task 8 is done after full system integration.

10. Appendix A:

1. **Hardware configurations**

   GPU**:** NVIDIA RTX 3060 / 4060 / 4070 (recommended) - Required for RT-DETR, SAM, and Zoe Depth fast inference
   CPU**:** Intel i5 / i7
   RAM**:** Minimum 16 GB (32 GB recommended for large batches)
   Storage**:** 50–80 GB for dataset + model outputs

2. **Platform/tools used**

   Programming Language: Python 3.8+
   Deep Learning Frameworks: PyTorch (for RT-DETR, SAM, ZoeDepth, Score-CAM) & Torch vision
   Libraries: NumPy, OpenCV, Matplotlib, Scipy (for Pearson correlation) & Pandas
   Models Used: RT-DETR (real-time object detection), SAM (object segmentation), Score-CAM (attention heatmaps) & Zoe Depth (depth estimation)

3. **Data description** The COCO 2017 Unlabelled Dataset was used to evaluate the framework in an annotation-free setting. A subset of 100 representative images was selected across **15 object categories** like Airplane, Bed, Elephant, Giraffe, Pizza, Person, Motorcycle, Parking Meter, Horse, Stop Sign, Zebra, Bear, Cake, Teddy Bear, and Hot Dog to cover diverse object categories and scene complexity while maintaining computational feasibility. These images were used to analyse the correlation between detection attention, segmentation masks, and depth representations within the integrated pipeline.
   Dataset Link: https://www.kaggle.com/datasets/awsaf49/coco-2017-dataset

4. **Model Architecture and hyper-parameters**

   The framework integrates multiple computer vision models into a unified pipeline to analyse the relationship between object detection, visual attention, segmentation, and depth estimation. RT-DETR performs real-time object detection and generates bounding boxes that act as spatial anchors for subsequent stages. Score-CAM produces attention maps highlighting image regions influencing detection decisions. These bounding boxes are used as prompts for the Segment Anything Model (SAM) to generate object-level segmentation masks. Zoe Depth then estimates monocular depth maps from the input image, allowing depth values to be extracted for segmented objects. Pearson correlation is applied to measure alignment between attention maps, segmentation masks, and depth representations.

Key hyper-parameters include detection confidence threshold, object query size, Score-CAM target layer, SAM bounding box prompts, and depth inference resolution.

5. **Training procedure**

The framework operates using pretrained deep learning models and does not require end-to-end training. RT-DETR is used as a pretrained object detection model to generate bounding boxes and confidence scores from input images. Score-CAM is applied as a post-hoc interpretability technique to produce attention maps without additional training. SAM generates object segmentation masks using bounding box prompts from RT-DETR, enabling annotation-free segmentation. Zoe Depth produces dense depth maps from single RGB images using pretrained weights. After generating these spatial representations, Pearson correlation is computed to analyse relationships between attention maps, segmentation masks, and depth values. Experiments are conducted on 100 representative images from the COCO 2017 dataset in an end-to-end inference pipeline.

6. Code Repository and Model Weights

The complete implementation of the VisualCue Framework is available on GitHub:
GitHub Repository: https://github.com/saransvijay/Project_UOA

Due to GitHub's file size limitation of 100 MB per file, the pretrained model weights (e.g., SAM ViT-B with size ~375 MB) could not be uploaded directly to the repository. To address this, the model weights are accessed from the official source:
https://dl.fbaipublicfiles.com/segment_anything/sam_vit_b_01ec64.pth
Download the weights manually and place them in the appropriate weights/ directory before running the application. This approach ensures that the repository remains lightweight while maintaining full reproducibility of the framework.

7. **Evaluation metrics**

Correlation Result:

```
...
    Processing image 1/100: 000000230735.jpg
    100%|██████████| 128/128 [01:13<00:00,  1.74it/s]

    Processing image 2/100: 000000513891.jpg
    100%|██████████| 128/128 [01:17<00:00,  1.65it/s]

    Processing image 3/100: 000000206107.jpg
    100%|██████████| 128/128 [00:53<00:00,  2.37it/s]

    Processing image 4/100: 000000505160.jpg
    100%|██████████| 128/128 [00:50<00:00,  2.51it/s]

    Processing image 5/100: 000000282515.jpg
    100%|██████████| 128/128 [00:56<00:00,  2.28it/s]

    Processing image 6/100: 000000359925.jpg
    100%|██████████| 128/128 [00:51<00:00,  2.50it/s]

    Processing image 7/100: 000000204930.jpg
    100%|██████████| 128/128 [00:51<00:00,  2.50it/s]

    Processing image 8/100: 000000432897.jpg
    100%|██████████| 128/128 [00:51<00:00,  2.49it/s]

    Processing image 9/100: 000000239930.jpg
    100%|██████████| 128/128 [00:51<00:00,  2.47it/s]

    Processing image 10/100: 000000421333.jpg
    100%|██████████| 128/128 [00:51<00:00,  2.48it/s]
```

.....

```
Processing image 92/100: 000000527615.jpg
100%|██████████| 128/128 [00:57<00:00,  2.22it/s]

Processing image 93/100: 000000215776.jpg
100%|██████████| 128/128 [00:52<00:00,  2.43it/s]

Processing image 94/100: 000000571160.jpg
100%|██████████| 128/128 [00:56<00:00,  2.28it/s]

Processing image 95/100: 000000435902.jpg
100%|██████████| 128/128 [00:50<00:00,  2.55it/s]

Processing image 96/100: 000000457037.jpg
100%|██████████| 128/128 [00:52<00:00,  2.45it/s]

Processing image 97/100: 000000110829.jpg
100%|██████████| 128/128 [00:56<00:00,  2.27it/s]

Processing image 98/100: 000000116229.jpg
100%|██████████| 128/128 [00:51<00:00,  2.50it/s]

Processing image 99/100: 000000348451.jpg
100%|██████████| 128/128 [00:52<00:00,  2.42it/s]

Processing image 100/100: 000000313520.jpg
100%|██████████| 128/128 [00:51<00:00,  2.48it/s]

FINAL MULTI-IMAGE RESULTS
Valid samples (Score-CAM vs SAM): 100
Valid samples (Score-CAM vs Depth): 100
Mean Score-CAM vs SAM: 0.1850 ± 0.0496
Mean Score-CAM vs Depth: 0.1425 ± 0.0131
2026-03-11 10:00:04.571509
```

Category Level Result:

```
FINAL CLASS RESULT
airplane Final Correlation : 0.388

FINAL CLASS RESULT
bed Final Correlation : 0.287

FINAL CLASS RESULT
elephant Final Correlation : 0.373

FINAL CLASS RESULT
Giraffe Final Correlation : 0.425

FINAL CLASS RESULT
pizza Final Correlation : 0.303

FINAL CLASS RESULT
person Final Correlation : 0.320

FINAL CLASS RESULT
motorcycle Final Correlation : 0.312

FINAL CLASS RESULT
parking meter Final Correlation : 0.355

FINAL CLASS RESULT
horse Final Correlation : 0.345

FINAL CLASS RESULT
stop sign Final Correlation : 0.335

FINAL CLASS RESULT
zebra Final Correlation : 0.347

FINAL CLASS RESULT
bear Final Correlation : 0.345

FINAL CLASS RESULT
cake Final Correlation : 0.312

FINAL CLASS RESULT
teddy bear Final Correlation : 0.292

FINAL CLASS RESULT
hot dog Final Correlation : 0.297
```